

BPMN 기반 비전문가용 스마트 컨트랙트 제어 도구 — 세부 구현 가이드

작성일 2026-09-16 · 대상 장진우 (단독 개발, AI 보조) · 가칭 BlockFlow 근거 중국어권 논문 16편·특허 3건, 영어권 논문/도구 20여 건, 2026-09 기준 스택 검증 (부록 F에 검증 상태 표기)

0. 이 문서를 읽는 법

이 가이드는 "BPMN 다이어그램을 그리면 스마트 컨트랙트가 되고, 그 다이어그램으로 컨트랙트를 계속 제어할 수 있는 도구"를 혼자서 4~5개월 안에 만들 수 있는 수준으로 짚은 설계 문서다.

장	내용	언제 읽나
1	문제 정의, "제어"의 정의, 비목표	처음
2	선행연구에서 가져온 설계 결정 (왜 이렇게 만드는가)	처음, 그리고 설계를 바꾸고 싶을 때
3	시스템 아키텍처	코드 시작 전
4	BPMN 서브셋 스펙 (L0/L1/L2) + 확장 속성 + 정형성 규칙	Modeler·Validator 구현 시
5	중간표현(IR) 스펙	Compiler 구현 시
6	Solidity 생성 규칙 + 검증된 예시 컨트랙트	Compiler 구현 시 (가장 중요)
7	검증 파이프라인	CI 구성 시
8	런타임·제어 UI	프론트엔드 구현 시
9	지갑·가스 추상화 (비전문가 UX의 핵심)	Phase 2
10	스택·버전표 (2026-09)	패키지 설치 시
11	로드맵 (Phase 0~4)	일정 잡을 때
12	리스크·오픈 이슈	수시로
부록	moddle 확장 JSON, IR 예시, 생성 Solidity 전문, 테스트, 中英韓 용어표, 참고문헌	필요 시

검증 표기 — 이 문서의 사실 주장에는 [읽음] (원문 직접 확인) / [초록] (초록·README만 확인) / [미검증] (서지사항만 확인) 을 붙였다. 부록 F 참고문헌 표에 전부 모아 두었다. 6장의 예시 컨트랙트는 **solc 0.8.37로 컴파일하고 JS EVM에서 17개 트랜잭션 시나리오를 실제 실행해 검증했다** (7.6절).

1. 문제 정의와 설계 목표

1.1 한 문장 정의

비전문가(프로그래밍·지갑·가스를 모르는 사람)가 **BPMN 다이어그램을 그리는 것만으로** 다자간 업무 프로세스를 스마트 컨트랙트로 배포하고, 같은 다이어그램 위에서 **인스턴스를 만들고, 자기 차례의 태스크를 완료하고, 진행 상태를 보고, 프로세스를 멈추거나 교체**할 수 있는 웹 도구.

1.2 "제어(control)"의 정의

선행연구 대부분은 **생성(generation)**에서 멈춘다. 이 도구가 차별화되는 지점은 생성 이후의 **제어 루프**다. 제어를 다음 6개 동작으로 고정한다.

#	제어 동작	누가	온체인 효과
C1	프로세스 배포	소유자	컨트랙트 1개 배포
C2	인스턴스 생성 + 역할 배정	소유자 (또는 허용된 역할)	<code>createInstance()</code>
C3	태스크 완료 (데이터 입력 포함)	해당 역할 담당자	<code>taskFn(id, args)</code>
C4	역할 담당자 교체	소유자	<code>rebindRole()</code>
C5	일시정지 / 재개	소유자	<code>setPaused()</code>
C6	프로세스 버전 교체	소유자 (+선택적 투표)	새 컨트랙트 배포 + 마이그레이션 정책

각 동작은 UI에서 **다이아그램 위의 클릭**으로 노출된다 (8장). 사용자는 "함수", "트랜잭션", "가스"라는 단어를 보지 않는다.

1.3 설계 목표 (우선순위 순)

1. **정확성** — 다이어그램의 의미 그대로 실행된다. 활성화되지 않은 태스크는 절대 실행되지 않고, 권한 없는 사람은 절대 완료할 수 없다.
2. **비전문가 UX** — 지갑 설치·시드 구문·가스 충전 없이 이메일/패스키 로그인만으로 쓴다.
3. **감사 가능성** — 모든 완료 이벤트가 온체인 로그로 남고, 다이어그램 위에 타임라인으로 재생된다.
4. **낮은 비용** — L2/Kaia 기준 태스크 1건 완료 $\leq 60k$ gas (측정치: 37k~60k, 6.8절).
5. **점진적 표현력** — L0 서브셋으로 시작해 L1, L2로 확장 가능하되 L0 프로세스는 언제나 동작.

1.4 비목표 (MVP에서 하지 않는 것)

- 임의의 Solidity 로직 삽입 (스크립트 태스크는 L1에서 제한된 DSL로만)
- 크로스체인, 프라이버시(ZK), 오프체인 상태채널 — 12장에 향후 과제로만 기록
- Hyperledger Fabric 타깃 — 중국 계열 연구의 절반이 Fabric이지만 (哈工大 ChainCollab, 北京物资学院), 비전문가 UX(지갑 추상화·퍼블릭 감사)는 EVM 계열이 훨씬 성숙하므로 EVM 우선. IR을 플랫폼 중립으로 설계해 Fabric 백엔드를 후속으로 붙일 수 있게 한다 (5장).

2. 선행연구에서 가져온 설계 결정

아래 표가 이 문서의 뼈대다. 각 결정은 "어떤 연구가 무엇을 보여줬는가 → 우리는 무엇을 채택하는가"로 읽는다.

#	설계 질문	근거 연구	채택 결정
D1	프로세스 상태를 온체인에 어떻게 저장?	García-Bañuelos et al. 2017 [읽음]: BPMN→Petri net→축약→ uint256 비트맵 marking , 2016년 방식 대비 실행가스 -22~25%. Caterpillar [읽음]·Chorpiller [읽음] 모두 동일 인코딩	시퀀스 플로우 1개 = 비트 1개 . <code>marking & inFlow != 0</code> 이 "태스크 활성화" 조건. 256개 플로우까지 컨트랙트 1개 (초과 시 <code>uint256[]</code>)
D2	컴파일형 vs 해석형?	Caterpillar v3 EDOC 2019 [읽음]: 해석형이 인스턴스 생성 가스 55~80% 절감, 대신 3.4M gas 인터프리터 1회 배포·가독성 저하. 俞东进 2021 (杭电) [읽음]: 편성 메타모델을 데이터로 저장 해 재배포 없이 프로세스 변경, 투표로 버전 승인. FlexChain 2023 [읽음]: 로직은 IPFS 규칙, 온체인엔 상태만	MVP는 컴파일형 (프로세스 1개 = 컨트랙트 1개, 인스턴스는 컨트랙트 내부 mapping). 이유: 생성 코드가 사람이 읽을 수 있어 감사·검증이 쉽고, 비트맵 덕에 이미 충분히 싸다. 버전 교체(C6)는 "새 컨트랙트 + 마이그레이션 정책"으로 처리. 해석형 엔진은 Phase 4 옵션
D3	인스턴스마다 컨트랙트를 배포?	Weber 2016·ChorChain 2020 [읽음]: 인스턴스 = 컨트랙트 → 인스턴스 생성 0.8~4.5M gas. 俞东进 2021: 범용 계약 + 인스턴스 0.21M gas	한 컨트랙트에 다중 인스턴스 (<code>mapping(uint => Instance)</code>). 측정: 인스턴스 생성 152k~169k gas (6.8절)

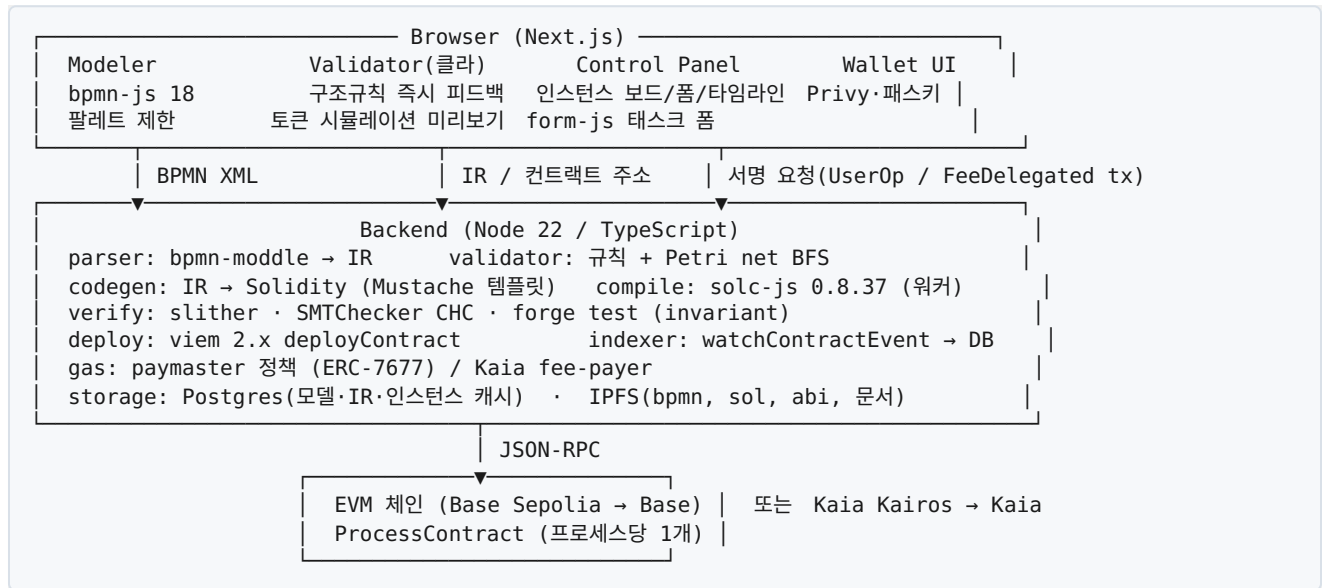
#	설계 질문	근거 연구	채택 결정
D4	오케스트레이션(단일 풀+레인) vs 코레오그래피?	ChorChain·Chorpiller: 코레오그래피는 다자간 메시지 교환에 자연스러움. 中山大 SCG4IOPC 2023 [읽음]: 협업 다이어그램(풀 여러 개)을 CSP#로 검증 후 Solidity. Stiehle & Weber SLR 2022 [읽음]: 두 계열의 장단 정리	레인 = 역할인 단일 풀 프로세스 로 시작. 비전문가는 "누가 무엇을 하는가"를 레인으로 가장 쉽게 표현하고, bpmn-js 18이 그대로 지원한다 (chor-js는 bpmn-js 7에 묶여 사실상 유지보수 중단 [읽음]). 다자간 메시지는 L1의 메시지 이벤트로 흡수
D5	역할 바인딩?	Weber 2016: 인스턴스 생성 시 공개키 등록. López-Pintado CAiSE 2019 [읽음]: 지명·승인 정책 DSL, 정책 배포 154k + 지명 111~168k gas. BlockCollab 2024 (哈工大) [읽음]: X.509 ABAC	생성 시 정적 바인딩 + 소유자 교체(rebind) . 동적 지명/승인 정책은 L2
D6	게이트웨이 조건은 어디서 평가?	Caterpillar: step() 안에 인라인. 2017: predicates 비트로 별도 저장	프로세스 변수 위의 작은 조건 DSL (4.4절)을 <code>_step()</code> 안 Solidity 식으로 컴파일. 변수는 태스크 완료 시 인자로 들어옴
D7	데이터는 온체인?	Weber 2016: 큰 데이터는 해시+URI. BlockCollab: 문서는 IPFS, DMN은 오프체인 엔진	값 타입(uint/bool/address/bytes32)만 온체인 , 문서는 bytes32 해시 (IPFS CID 앞 32바이트 또는 keccak). DMN/결정표는 L2
D8	LLM을 코드 생성에 쓰나?	TUM 2025 [읽음]: 7개 LLM으로 165개 코레오그래피 생성, 최고 F1 0.918 → "불변 컨트랙트에 92%는 부적합, 템플릿/테스트에만 쓰라". 武汉大 FSM-SCG 2025 [읽음]: 자연어→FSM 중간모델→LLM으로 컴파일 성공률 +48%. arXiv 2602.04039 [읽음]: LLM 생성 컨트랙트에 반복적 보안 결합	코드 생성은 규칙 기반 템플릿만 . LLM은 (a) 자연어→BPMN 초안, (b) 다이어그램 자연어 설명, (c) 테스트 케이스 초안에만. FSM-SCG의 "구조화 중간모델을 먼저" 원칙은 IR 설계에 반영
D9	검증은 어떻게?	北京物资学院/北航 2024 [읽음]: BPMN→Prolog, 8개 구조 규칙. 中山大: CSP# 모델체킹으로 데드락·미도달 검출, 5개 중 2개 사례가 초기 검증 실패. solc SMTChecker CHC [읽음]	4단계 파이프라인 (7장): 구조 규칙 → Petri net 도달성(soundness) → Slither + SMTChecker(CHC) → 자동 생성 Foundry 불변식 테스트
D10	비전문가 지갑/가스?	(연구 아닌 인프라) EIP-7702 Pectra 2025-05 활성화 [읽음], ERC-4337 EntryPoint v0.7/0.8/0.9 [읽음], Kaia 프로토콜 네이티브 수수료 대납 [읽음]	임베디드 지갑(패스키) + 가스 스폰서 . EVM L2 경로 (Base + paymaster)와 Kaia 경로(fee delegation) 둘 다 설계, MVP는 하나만 (9장)
D11	모델러?	bpmn-js 18.28 [읽음] (위터마크 유지 조건 MIT형 라이선스), 토큰 시뮬레이션 0.40 [읽음] bpmn-js 18 지원, form-js 1.26 [읽음]	bpmn-js + 팔레트 제한 + moddle 확장 + 토큰 시뮬레이션(미리보기) + form-js(태스크 입력 폼)
D12	중국 산업계 참고	中央财经大 특허 CN117234488B [읽음]: EPC→EPML(XML)→키워드 매핑→Solidity. 墨石科技 CN112148278A [읽음]: 드래그앤드롭 데이터모델/역할/조건→체인코드+API 자동생성. 平安 CN112015396A [읽음]: DSL→ANTLR→템플릿 렌더링	"XML 중간표현 → 키워드 매핑 → 템플릿 렌더링"이라는 3단 구조가 산업계 특허에서도 반복됨 → 5·6장 파이프라인과 일치. 墨石의 "역할이 어떤 조건에서 어떤 필드에 어떤 연산"은 4.3절 확장 속성 설계에 반영

2.1 앵커 논문 두 편의 위치

- 《面向信用服务的BPMN智能合约范式研究》 薄钧戈, 西安交通大学, 《信息安全研究》 2022, 8(5): 475–483 [미검증: 서지사항만 온라인 확인, 본문은 sicris.cn 리다이렉트 루프로 접근 불가]. 이미 문장 단위 스터디 노트를 만들어 두었으므로, 이 논문의 "범식(范式)" — 신용 서비스 도메인에서 BPMN 요소를 계약 구성요소로 대응시키는 틀 — 을 **4.3절 확장 속성의 도메인 프리셋(신용/여신 템플릿)**으로 구체화하는 것이 가장 자연스러운 활용이다. 노트의 매핑 표를 4.3절 표 형식으로 옮겨 넣을 것.
- 《SmartGen-AADL》 《软件学报》 2026 제8기 "大模型与软件工程" 특집 우선출판 [미검증: 특집 게재 사실만 확인, 저자·소속·본문 미확인]. 이름대로 AADL 아키텍처 모델 + LLM 생성이라면, 이 도구에서의 대응물은 **"IR(구조화 모델)을 먼저 확정하고 LLM은 보조"**라는 D8 결정이다. 본문을 확보하면 IR 설계(5장)와 비교표를 추가할 것.

3. 시스템 아키텍처

3.1 컴포넌트



3.2 데이터 흐름 (한 프로세스의 일생)

- 그리기** — 사용자가 제한된 팔레트로 다이어그램을 그린다. 각 태스크에 역할(레인)·입력 필드·조건을 속성 패널에서 채운다 → `saveXML()`.
- 검증** — 클라이언트에서 구조 규칙 즉시 표시(빨간 배지), 서버에서 Petri net 도달성 검사. 통과하면 토큰 시뮬레이션 "미리보기" 버튼 활성화.
- 컴파일** — XML → IR(JSON) → Solidity. IR과 Solidity 둘 다 IPFS에 저장, CID를 DB에.
- 정적 검증** — solc 컴파일 + Slither + SMTChecker. 경고는 사용자에게 "다이어그램 문제"로 번역해서 보여준다 (예: "승인 태스크 뒤에 도달할 수 없는 경로가 있어요").
- 배포** — 사용자의 스마트 계정이 (가스 스폰서로) 배포 트랜잭션 전송. 소유자 = 그 계정.
- 제어** — 인스턴스 생성/역할 배정/태스크 완료. 인덱서가 이벤트를 DB에 적재, UI가 다이어그램 위에 현재 marking을 색칠.
- 감사** — TaskCompleted / MarkingChanged 이벤트를 타임라인으로. 블록 익스플로러 링크는 "고급" 아래에 숨긴다.

3.3 신뢰 관계

- 온체인 컨트랙트만이 **정확성의 근거**다. 백엔드·인덱서는 편의 계층이며, 다운돼도 익스플로러+ABI로 제어 가능해야 한다 (감사 가능성 목표).
- 가스 스폰서(paymaster / fee payer)는 **정책으로 함수 allowlist**를 걸어 남용을 막는다 (9.4절).

4. BPMN 서브셋 스펙

4.1 레벨 정의

레벨	요소	근거
L0 (MVP)	시작 이벤트(none), 종료 이벤트(none), 사용자 태스크 , 배타(XOR) 게이트웨이 분기/합류, 병렬(AND) 게이트웨이 분기/합류, 시퀀스 플로우(조건식·기본 플로우), 레인(=역할) , 프로세스 변수(확장 속성)	García-Bañuelos 2017 지원 집합 + 레인. 北京物资学院 2024는 병렬을 뺐지만, 비트맵 인코딩에서 AND는 공짜에 가까움

레벨	요소	근거
L1	서비스 태스크(오라클 콜백: 계약이 이벤트를 내고 오프체인이 응답), 타이머 경계 이벤트(block.timestamp 기한 → 만료 경로), 메시지 시작/중간 이벤트(외부 주소가 보내는 트랜잭션), 결제 태스크(ERC-20 전송/에스크로, Weber 2016의 payment task), 루프(루프백 시퀀스 플로우 — 1-safe 유지 조건 하), 포괄(OR) 게이트웨이(Pupa 2022 방식)	Caterpillar·Pupa·BlockCollab
L2	임베디드 서브프로세스(인라인 전개), 콜 액티비티(별도 컨트랙트), 다중 인스턴스(HIT 2025 Statecharts 방식 검토), 코레오그래피 뷰, DMN 결정표(오프체인 엔진 + 온체인 해시 검증), 동적 역할 지명/승인 정책	Caterpillar v2.1/v3, BlockCollab, 哈工大 2025

L0의 형식 제약: 프로세스는 **1-safe 워크플로우 넷** 이어야 한다 (모든 플로우에 토큰이 최대 1개). 이 제약이 있어야 비트맵 인코딩이 정확하다. 검증기가 강제한다 (7.2절).

4.2 팔레트 제한 (bpmn-js)

기본 `PaletteProvider` 를 교체해 다음만 남긴다: 손 도구, 선택, 시작 이벤트, 종료 이벤트, 사용자 태스크, XOR 게이트웨이, AND 게이트웨이, 레인 추가. `bpmn-js-create-append-anything` 모듈은 **포함하지 않는다** (컨텍스트 메뉴로 다른 요소가 새어 들어오는 것을 막기 위해). 레인은 팔레트 대신 "역할 추가" 버튼으로 노출 (내부적으로 `bpmn:Lane` 생성).

4.3 확장 속성 (moddle extension, 네임스페이스 bc)

bc: 네임스페이스로 다음 속성을 정의한다 (부록 A에 JSON 전문).

대상 요소	속성	타입	의미
bpmn:Process	bc:variables	bc:Variable[]	프로세스 변수 선언 {name, type ∈ {uint256,int256,bool,address,bytes32,string(해시로 저장)}, initial}
bpmn:Lane	bc:roleKey	String	역할 상수 이름 (예: Manager). 비어 있으면 레인 이름에서 파생
bpmn:Lane	bc:bindingMode	static \ ownerRebind \ open	open 이면 누구나 처음 완료하는 주소가 역할에 바인딩 (ChorChain의 선택적 역할 구독 참고)
bpmn:UserTask	bc:inputs	bc:Input[]	완료 시 입력받는 값 {variable, label, required} → 함수 인자 + form-js 필드
bpmn:UserTask	bc:taskId	Integer	이벤트/UI용 정수 ID (생성기가 자동 부여)
bpmn:SequenceFlow	conditionExpression (표준)	bc:expr 언어	4.4절 DSL
bpmn:ExclusiveGateway	default (표준)	flow ref	기본 플로우 필수 (조건이 모두 거짓일 때 토큰 소실 방지)
bpmn:UserTask	bc:payment (L1)	{token, to: role\ address, amountVar}	결제 태스크
bpmn:BoundaryEvent (timer, L1)	bc:deadlineVar	variable	block.timestamp >= startedAt + deadline

도메인 프리셋: 앵커 논문(薄钧戈 2022)의 신용 서비스 매핑을 `presets/credit-service.json` 으로 만들어 "신용 심사 템플릿" 으로 팔레트 옆에 노출한다 (변수·역할·태스크 세트 미리 채움).

4.4 조건식 DSL (bc:expr)

비전문가가 속성 패널의 드롭다운으로 조립하고, 내부적으로 문자열로 저장한다. 문법 (EBNF):

```

expr      := or ;
or         := and { "|" and } ;
and        := cmp { "&&" cmp } ;
cmp        := term ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) term | "!" cmp | "(" or ")" | boolvar ;
term       := variable | integer | "true" | "false" | roleaddr ;
variable:= identifier          (* bc:variables 에 선언된 이름만 *)
roleaddr:= "role(" identifier ")" (* 역할 주소 비교용 *)

```

- 타입 검사: uint256 은 정수와만, bool 은 == / != /단독, address 는 role(...) /리터럴과만.
- 컴파일: 그대로 Solidity 식으로 변환 (amount > 1000 → v.amount > 1000). 나눗셈·곱셈은 L0에서 제외 (오버플로/정밀도 설명 부담).
- UI 표현: "만약 [금액] 이 [보다 큼] [1000]" 식의 자연어 빌더.

4.5 정형성 규칙 (Validator, 클라이언트 즉시 검사)

北京物资学院/北航 2024의 8개 구조 규칙 [읽음] 을 L0에 맞게 확장한 12개.

#	규칙	오류 메시지(비전문가용)
R1	시작 이벤트 정확히 1개, 들어오는 플로우 없음	"시작점은 하나여야 해요"
R2	종료 이벤트 ≥ 1개, 나가는 플로우 없음	"끝나는 지점이 필요해요"
R3	모든 태스크는 들어오는 플로우 1개, 나가는 플로우 1개 (분기는 게이트웨이로만)	"태스크에서 갈라지려면 마름모(게이트웨이)를 쓰세요"
R4	게이트웨이는 분기(in 1, out ≥2) 또는 합류(in ≥2, out 1) 중 하나	"이 마름모는 갈라지거나 모으는 것 중 하나만 해야 해요"
R5	XOR 분기의 모든 나가는 플로우에 조건 있음, 단 기본 플로우 1개 필수	"조건이 없는 화살표가 있어요 / 기본 화살표를 정하세요"
R6	AND 분기 플로우에는 조건 없음	"동시에 진행되는 화살표엔 조건을 붙일 수 없어요"
R7	모든 태스크는 레인 안에 있음	"이 일은 누가 하나요? 역할 칸 안으로 옮기세요"
R8	시작→모든 노드 도달 가능, 모든 노드→어떤 종료 도달 가능 (그래프 수준)	"도달할 수 없는 일이 있어요"
R9	조건식 타입 검사 통과, 참조 변수 선언됨	"조건에 쓰인 [x]는 아직 정의되지 않았어요"
R10	조건에 쓰이는 변수는 그 게이트웨이 이전의 어떤 태스크 bc:inputs 에서 반드시 설정됨 (모든 경로)	"[금액]을 입력받기 전에 조건에서 쓰고 있어요"
R11	시퀀스 플로우 수 ≤ 256 (L0)	"프로세스가 너무 커요. 나눠 주세요"
R12	요소 이름 비어 있지 않음 (함수명·이벤트 라벨 생성용)	"이름을 붙여 주세요"

R8 은 그래프 도달성, 진짜 **soundness(데드락·1-safe)**는 서버의 Petri net 검사 (7.2절) 에서 본다.

5. 중간표현(IR) 스펙

IR 은 플랫폼 중립 JSON 이다. 목적: (1) 코드 생성기가 BPMN XML 을 다시 파싱하지 않게, (2) Fabric/Go 백엔드를 나중에 붙일 수 있게, (3) 검증기·시뮬레이터·UI 가 같은 모델을 쓰게.

5.1 스키마 (요약)

```

{
  "version": "bf-ir/0.1",
  "process": { "id": "ExpenseApproval", "name": "경비 승인", "sourceCid": "bafy..." },
  "roles": [ { "key": "Requester", "label": "신청자", "binding": "static" }, ... ],
  "variables": [ { "name": "amount", "type": "uint256" }, { "name": "approved", "type": "bool" },

```

```

        { "name": "receiptHash", "type": "bytes32" } ],
    "nodes": [
        { "id": "start", "kind": "startEvent" },
        { "id": "T1", "kind": "userTask", "taskId": 1, "name": "submit", "label": "경비 신청",
          "role": "Requester", "inputs": [ { "variable": "amount", "label": "금액" } ],
          "in": ["F1"], "out": ["F2"] },
        { "id": "X1", "kind": "xorSplit", "in": ["F2"],
          "branches": [ { "flow": "F3", "cond": "amount > 1000" } ], "default": "F4" },
        { "id": "A1", "kind": "andSplit", "in": ["F4", "F11"], "out": ["F6", "F7"] },
        { "id": "A2", "kind": "andJoin", "in": ["F8", "F9"], "out": ["F10"] },
        { "id": "end_ok", "kind": "endEvent", "in": ["F10"], "outcome": "completed" },
        { "id": "end_rej", "kind": "endEvent", "in": ["F12"], "outcome": "rejected" }
    ],
    "flows": [ { "id": "F1", "bit": 0, "from": "start", "to": "T1" }, ... ],
    "silent": ["X1", "X2", "A1", "A2", "end_ok", "end_rej"] // _step() 에서 처리되는 침묵 전이
}

```

5.2 변환 규칙 (BPMN → IR)

- 비트 배정:** 시퀀스 플로우를 문서 순서로 정렬해 0부터 부여. 결정적(deterministic)이어야 재컴파일 시 동일 결과.
- 합류 게이트웨이 정규화:** XOR 합류는 IR 노드를 만들지 않고 들어오는 플로우들을 다음 노드의 in 집합으로 병합 (García-Bañuelos 2017의 τ -전이 제거 규칙의 단순화). AND 합류만 노드로 남긴다.
- AND 분기의 in 이 여러 개**(위 A1처럼 XOR 합류가 앞에 있던 경우는 "어느 하나라도 있으면 소비"로 컴파일 ($m \& (F4|F11) \neq 0$)). 1-safe 검증이 있어야 안전.
- 종료 이벤트:** 여러 개면 outcome 라벨을 붙여 InstanceEnded(id, outcomeCode) 로 구분.
- 침묵 전이 순서:** _step() 루프에서 XOR → AND split → AND join → End 순으로 배치. 루프는 고정점까지 돌므로 순서는 정확성이 아니라 가스에만 영향.

5.3 Petri net 뷰

IR 에서 곧바로 place = flow, transition = node 인 워크플로우 넷을 얻는다. 검증기(7.2절)와 토큰 시뮬레이터가 이 뷰를 공유한다.

6. Solidity 코드 생성 규칙

6.1 컨트랙트 골격

프로세스 1개 → 컨트랙트 1개. 구조는 고정된 6개 구역이며, 템플릿(Mustache)의 각 구역이 IR 의 어느 부분에서 채워지는지 아래 표에 적었다.

구역	내용	IR 출처
역할 상수	bytes32 constant ROLE_X = keccak256("X")	roles
플로우 비트 상수	uint256 constant F_n = 1 << bit (주석에 from→to)	flows
변수 구조체	struct Vars { ... } + mapping(uint => Vars)	variables
인스턴스 관리	Instance{marking, ended, creator}, createInstance(address[N]), rebindRole, setPaused	roles 개수
태스크 함수	userTask 마다 1개 (6.3절)	nodes[kind=userTask]
엔진	_fire(), _step(), enabledTasks()	silent

의존성 없음: MVP 생성 코드는 OpenZeppelin 을 import 하지 않는다 (소유자·일시정지 20줄을 인라인). 이유: 사용자가 익스플로러에서 검증(verify)할 때 단일 파일이 압도적으로 쉽고, 감사 대상이 작다. L1 결제 태스크부터 IERC20 인터페이스만 인라인.

6.2 상태 인코딩

```
uint256 marking; // bit i == 1 ⇔ 플로우 i 에 토큰 있음
// 태스크 활성화: marking & F_in != 0
// 태스크 완료: marking = (marking & ~F_in) | F_out
// AND split: marking = (marking & ~F_in) | (F_o1 | F_o2 | ...)
// AND join: if (marking & (F_i1|F_i2) == (F_i1|F_i2)) marking = (marking & ~(F_i1|F_i2)) | F_out
// XOR split: marking &= ~F_in; marking |= cond1 ? F_b1 : cond2 ? F_b2 : F_default
```

6.3 사용자 태스크 → 함수

```
/// T{n}: {label} [{role}] in: {F_in} out: {F_out} sets: {inputs}
function {name}(uint256 id[, type var]...)
    external whenNotPaused onlyRole(id, ROLE_{role})
{
    _require(id, F_in, TASK_{n}); // 미종료 + 활성화 검사 (커스텀 에러)
    vars[id].{var} = {var}; ... // 입력 저장
    _fire(id, F_in, F_out, TASK_{n}); // 토큰 이동 + 침묵 전이 + 이벤트
}
```

- 함수명은 bc:taskId 와 영문 name 에서 생성. 이름 충돌 시 _2 접미.
- string 입력은 받지 않는다 — 클라이언트가 keccak/IPFS CID 로 바꿔 bytes32 로 보낸다 (D7).
- msg.sender 는 **스마트 계정 주소** (9장). 역할 바인딩도 그 주소로.

6.4 이벤트

이벤트	용도
InstanceCreated(id, creator)	인스턴스 목록
RoleBound(id, role, account)	담당자 표시
TaskCompleted(id, taskId, actor)	타임라인
MarkingChanged(id, marking)	다이어그램 색칠 — 인덱서는 이 하나만 보면 현재 상태 복원 가능
InstanceEnded(id, outcome)	종료/결과

6.5 접근 제어와 안전장치

- onlyRole(id, ROLE) — 인스턴스별 역할 주소와 msg.sender 일치.
- whenNotPaused — 모든 태스크·인스턴스 생성에.
- AlreadyEnded — 종료 후 어떤 태스크도 불가.
- 재진입: 외부 호출이 없으므로 L0에서는 불필요. L1 결제 태스크에서 nonReentrant 인라인 + checks-effects-interactions.
- tx.origin 사용 금지 (EIP-7702 이후 EOA 가정 무효 [읽음]).

6.6 검증된 예시: "경비 승인" 프로세스

다이어그램 (레인: 신청자 / 팀장 / 재무):

```
[시작] → (신청자) 경비 신청{amount}
→ ◇X1 amount>1000? -yes→ (팀장) 승인{approved} → ◇X2 approved? -yes→ 1
                                     |default→ ◆A1
                                     |no(default)→ [종료:반려]
◆A1 → (재무) 지급 → ◆A2 → [종료:완료]
    ↳ (신청자) 영수증 제출{hash} ↳
```

플로우 비트: F1..F12 (부록 C 주석 참조). 이 다이어그램에서 생성되는 컨트랙트 전문이 **부록 C**이며, 다음을 확인했다.

- solc **0.8.37**, optimizer 200, evmVersion cancun: 경고 0, 바이트코드 3,960 B.
- JS EVM(@ethereumjs/vm 10) 에서 17개 트랜잭션 실행 (7.6절 로그). 활성화 전 실행·권한 없는 실행·종료 후 실행이 모두 커스텀 에러로 거부되고, 3개 인스턴스(승인 경로 / 소액 우회 경로 / 반려 경로)가 기대대로 종료.

6.7 템플릿 구조 (생성기 코드 배치)

```
codegen/
  templates/
    contract.sol.mustache      # 6.1 골격
    partials/roles.mustache
    partials/flows.mustache
    partials/task.mustache     # 6.3
    partials/step.mustache     # 침묵 전이 블록들 (xorSplit/andSplit/andJoin/end)
emit.ts                       # IR → 템플릿 컨텍스트 (비트 상수 문자열, 조건식 컴파일)
expr.ts                       # bc:expr → Solidity 식 (4.4)
names.ts                      # 한글 라벨 → 안전한 식별자, 충돌 처리
```

Chorpiler [읽음] 가 같은 Mustache + 비트배열 방식으로 Solidity 와 Algorand TEAL 두 타깃을 내므로, 구조 참고용으로 가장 가깝다.

6.8 가스 측정치 (예시 컨트랙트, JS EVM, Cancun)

동작	gas	비고
배포	905,046	프로세스당 1회
createInstance (역할 3)	152,026 ~ 169,126	첫 인스턴스가 약간 큼 (슬롯 초기화)
태스크 완료 (변수 저장 포함)	59,868 ~ 60,008	submit / approve / uploadReceipt
태스크 완료 (저장 없음)	37,190	pay
거부된 호출	26,197 ~ 30,627	커스텀 에러 revert

문헌 대비: Caterpillar 컴파일형 인스턴스 생성 1.1~2.8M, ChorChain 4.5M 배포 + 인스턴스=배포, 俞东进 2021 인스턴스 0.21M. **다중 인스턴스 단일 컨트랙트(D3)** 가 인스턴스 생성 비용을 한 자릿수 줄인다. Base 기준 (중앙값 수수료 \$0.002 [읽음]) 태스크 1건은 사실상 0원.

7. 검증 파이프라인

7.1 단계 개요

단계	시점	도구	실패 시 사용자에게
V1 구조 규칙 R1~R12	그리는 중 (클라)	자체 TS	요소에 빨간 배지 + 문장
V2 Soundness	저장/컴파일 요청 시 (서버)	자체 Petri net BFS	문제 경로를 다이어그램에 하이라이트
V3 정적 분석	컴파일 후	solc + Slither 0.11.6 + SMTChecker(CHC)	"고급" 탭에 원문, 요약은 번역
V4 자동 테스트	배포 전	Foundry 1.8 invariant + 생성된 시나리오 테스트	배포 버튼 비활성
V5 미리보기	언제나	bpmn-js-token-simulation 0.40	사용자 스스로 흐름 확인

7.2 V2 — Soundness 검사 (핵심 알고리즘)

1-safe 워크플로우 넷 + 비트맵이므로 상태 = uint256 하나. 조건식은 **비결정적 선택으로 추상화** (XOR 분기는 모든 가지를 탐색). BFS:

```
visited = {initialMarking}; queue = [initialMarking]
while queue:
    m = pop()
    for each transition t enabled in m:
        m' = fire(t, m)
        if (m' has bit that was already set in m for a produced flow) → 1-safe 위반 보고
        if m' not in visited: add, push
```

검사:

- 모든 도달 marking에서 (ended 이거나 활성 전이가 ≥ 1) → 아니면 데드락
- 종료 시 marking == 0 → 아니면 "남은 토큰"(proper completion)
- 모든 전이가 어떤 도달 marking에서 활성 → 아니면 dead task

L0 프로세스는 플로우 ≤ 256 , 실제 상태 수는 수백~수천이라 밀리초 안에 끝난다. 中山大가 CSP#/PAT로 하던 데드락·미도달 검출을 이 크기에서는 자체 BFS로 대체할 수 있다. L2 (다중 인스턴스)로 가면 哈工大 2025의 Statecharts 접근이 필요하다.

7.3 V3 — SMTChecker 설정

```
"settings": { "modelChecker": {
  "engine": "chc", "solvers": ["z3"],
  "targets": ["assert", "underflow", "overflow", "outOfBounds"],
  "invariants": ["contract"], "showUnproved": true, "timeout": 20000 } }
```

생성기가 `_fire()` 끝에 `assert(popcount(m) <= maxTokens)` 같은 검사 가능한 단언을 (테스트 빌드에서만) 삽입한다. BMC 엔진은 0.8.37에서 폐기 예고 [읽음] 이므로 CHC만 쓴다.

7.4 V4 — 자동 생성 Foundry 테스트

IR에서 다음 테스트를 생성한다 (부록 D에 예시):

- **불변식** `invariant_noGhostTokens`: 모든 인스턴스에서 marking의 set bit는 IR의 플로우 집합 안.
`invariant_endedMeansEmpty`: `ended` $\Rightarrow$ `marking == 0`.
- **핸들러**: 무작위 역할 계정이 무작위 태스크를 호출. `fail_on_revert = false`로 거부가 정상임을 허용.
- **시나리오**: V2의 BFS가 만든 각 도달 경로 하나당 1개 테스트 (경로 커버리지 100% 보장).

7.5 V3 결과의 "번역"

Slither 검출기 $\rightarrow$ 비전문가 문장 매핑표를 유지한다. 생성 코드는 고정 골격이라 나올 수 있는 경고가 한정적이다. 예상 밖 경고는 **템플릿 버그**로 취급해 배포를 막는다.

7.6 예시 컨트랙트 실행 로그 (검증 기록)

deploy ok= true gas= 905046			
createInstance	ok=true gas=169126	RoleBound×3 InstanceCreated(1)	
MarkingChanged(1,1)			
approve before submit (X)	ok=false gas=30627	TaskNotEnabled["1","2"]	
submit by stranger (X)	ok=false gas=26197	NotAuthorized["1",ROLE_REQUESTER]	
submit amount=5000	ok=true gas=59868	TaskCompleted(1,1) MarkingChanged(1,4)	#
F3 활성			
pay before approve (X)	ok=false gas=30415	TaskNotEnabled["1","3"]	
approve=true	ok=true gas=60008	TaskCompleted(1,2) MarkingChanged(1,96)	#
F6 F7			
uploadReceipt	ok=true gas=59904	TaskCompleted(1,4) MarkingChanged(1,288)	#
F6 F9			
pay	ok=true gas=37190	TaskCompleted(1,3) InstanceEnded(1,true)	
MarkingChanged(1,0)			
pay again (X, ended)	ok=false gas=28263	AlreadyEnded["1"]	

```
-- 인스턴스 2 (소액, 승인 생략) --
submit amount=300          ok=true  gas=59910  MarkingChanged(2,96)          #
X1 default → A1
pay / uploadReceipt        ok=true          InstanceEnded(2,true) | MarkingChanged(2,0)
-- 인스턴스 3 (반려) --
submit amount=9000 → approve=false          InstanceEnded(3,false) | MarkingChanged(3,0)
pay after reject (X)      ok=false          AlreadyEnded["3"]
```

8. 런타임·제어 UI

8.1 화면 4개 (비전문가 어휘로)

화면	보이는 것	온체인 연결
내 프로세스	프로세스 카드(이름, 버전, 진행 중 n건, 일시정지 토글)	paused, 인덱서 집계
프로세스 보드	다이어그램 위에 인스턴스별 토큰 표시(현재 marking을 플로우에 점으로), 활성 태스크는 펄스 애니메이션. 오른쪽에 인스턴스 리스트	MarkingChanged
할 일	내 역할에 활성화된 태스크 카드. 클릭 → form-js 폼(bc:inputs) → "완료" 버튼	enabledTasks(id) + taskFn
기록	타임라인 (누가·언제·무엇을), 다이어그램 재생 슬라이더	TaskCompleted 시계열

8.2 다이어그램 위 제어 매핑 (C1~C6)

- 시작 이벤트 클릭 → "새 건 시작" (C2): 역할별 담당자 선택 모달 (이메일 초대 → 스마트 계정 주소).
- 활성 태스크 클릭 → 폼 → 완료 (C3).
- 레인 헤더 클릭 → 담당자 교체 (C4).
- 프로세스 헤더의 일시정지 (C5).
- "새 버전 배포" (C6): 진행 중 인스턴스는 옛 컨트랙트에서 끝내고, 새 인스턴스만 새 컨트랙트로 (기본 정책). 옵션: 俞东进 2021 식 참여자 투표 후 전환.

8.3 인덱서

- `viem.watchContractEvent` (WebSocket 있으면 `eth_subscribe`, 없으면 폴링 [읽음]). 재시작 시 `fromBlock` 부터 재생.
- DB 테이블: `processes`, `instances(marking, ended, outcome)`, `events`, `role_bindings`.
- 인덱서 다운 시 UI는 `enabledTasks()` / `instances()` 직접 `readContract` 로 폴백 (3.3 신뢰 경계).

8.4 트랜잭션 전 시뮬레이션

완료 버튼을 누르기 전에 `simulateContract` 로 미리 실행해 커스텀 에러를 잡아 "이 일은 아직 차례가 아니에요" 처럼 보여준다. 실패 트랜잭션에 가스를 쓰지 않는다.

9. 지갑·가스 추상화

9.1 원칙

사용자는 (1) 이메일/패스키로 로그인하고, (2) "완료" 버튼만 누른다. 서명·가스·주소는 도구가 처리한다. 두 경로 중 하나를 MVP 로 택한다.

9.2 경로 A — EVM L2 (Base) + 스마트 계정 + Paymaster

- 임베디드 지갑: Privy (@privy-io/react-auth 3.43, 패스키·네이티브 가스 스폰서, ≤500 MAU 무료 [읽음]) 또는 Coinbase Base Account (패스키 네이티브, ERC-7677 paymaster URL, CDP 무료 크레딧 [읽음]).
- 계정: EIP-7702 위임 (Pectra 2025-05-07 활성화 [읽음]) — 기존 EOA 주소 그대로 배칭·스폰서 가능. 또는 ERC-4337 EntryPoint v0.7 (0x0000000071727De22E5E9d8BAf0edAc6f37da032) / v0.8 / v0.9 [읽음] + permissionless.js 0.4.
- 스폰서 정책: paymaster 가 우리 **ProcessContract** 주소 + **태스크 함수 선택** 만 허용. 인스턴스 생성·배포는 소유자 계정만.
- 테스트넷: Base Sepolia (chain 84532 [읽음]).

9.3 경로 B — Kaia + 프로토콜 네이티브 수수료 대납 (한국 컨텍스트)

- Kaia 는 TxTypeFeeDelegatedSmartContractExecution (0x31) · ...Deploy (0x29) 등 **수수료 대납 트랜잭션 타입**이 프로토콜에 있다 [읽음]. 사용자가 서명 → 우리 fee payer 가 공동 서명 → `kaia_sendRawTransaction`. 부분 대납(fee ratio) 도 가능.
- 관리형 대납 서비스: <https://fee-delegation.kaia.io> (테스트넷 -kairos), 컨트랙트/주소 화이트리스트, dApp 이 잔액 충전 [읽음].
- 라이브러리: @kaiachain/viem-ext 2.1 / @kaiachain/ethers-ext 2.1 (EIP-2718 타입이 아니라 필수 [읽음]).
- 가스 추상화(KIP-247/245, 화이트리스트 ERC-20 로 가스 지불) 는 메인넷 활성화 일자 미확인 [미검증].
- 테스트넷: Kairos (chain 1001, <https://public-en-kairos.node.kaia.io>).
- 장점: 4337 번들러·paymaster 스택 없이 백엔드 하나로 끝남. 단점: 임베디드 지갑 제공자의 Kaia 지원 여부를 확인해야 함 (Privy 는 EVM 체인 임의 추가 가능하나 대납 타입 서명은 커스텀 필요).

9.4 남용 방지

- 스폰서 정책: 함수 allowlist + 계정당 일일 한도 + `simulateContract` 성공한 호출만 릴레이.
- 역할 초대: 초대 링크 → 로그인 → 스마트 계정 주소 확보 → 소유자가 `createInstance` 시 배정. 담당자가 사라지면 `rebindRole`.

9.5 권장

한국 사용자·기관 대상 데모라면 **B(Kaia)** 가 운영 부담이 작다. 글로벌/오픈소스 지향이면 **A(Base)**. IR·생성기는 두 경로에 동일하고 배포·서명 어댑터만 다르므로, 어댑터 인터페이스(`deploy()`, `sendTask()`, `watch()`)를 먼저 고정한다.

10. 기술 스택·버전표 (2026-09-16 검증)

영역	선택	버전	비고
모델러	bpmn-js	18.28.0	Node ≥ 20, ESM. 워터마크 유지 조건 라이선스
	bpmn-moddle	10.2.0	bc 확장
	bpmn-js-properties-panel / @bpmn-io/properties-panel	5.65.0 / 3.55.0	커스텀 PropertiesProvider
	bpmn-js-token-simulation	0.40.0	미리보기
	@bpmn-io/form-js	1.26.0	태스크 폼
컴파일	solc-js	0.8.37	서버는 워커 스레드, 브라우저는 Web Worker 필수 (web-solc 0.7)
	Foundry	1.8.3	forge test/invariant, anvil 로컬

영역	선택	버전	비고
	Hardhat 3 (선택)	3.16.0	Node $\geq$ 22.13
체인 접근	viem / wagmi	2.56.5 / 3.7.7	
AA	permissionless.js	0.4.1	EntryPoint v0.7
	Privy / Base Account	3.43.0 / 2.5.10	패스키
Kaia	@kaiachain/viem-ext	2.1.0	대납 tx 타입
검증	Slither	0.11.6	Hardhat 3 지원
	SMTChecker	solc 내장	CHC 만
	Echidna / Medusa (선택)	2.3.2 / 1.5.1	
백엔드	Node / TypeScript / Postgres / IPFS(kubo 또는 핀 서비스)	Node 22	
프론트	Next.js + Tailwind	—	

라이선스 주의: bpmn-js·form-js 는 워터마크 유지 조건. 공식 제거 옵션은 확인되지 않음 [읽음].

11. 로드맵 (단독 개발 기준, 주 단위)

Phase	주	산출물	완료 기준
0 골격	1-2	리포 구조, IR 스키마(JSON Schema), 예시 IR 3개 손으로 작성, 6.6 예시 컨트랙트를 템플릿에서 재생성해 부록 C 와 diff 0	pnpm test 에서 codegen 스냅샷 통과
1 컴파일러	3-5	bpmn-moddle 파서 → IR, V1 규칙, V2 BFS, Mustache 생성기, solc 컴파일, Foundry 테스트 생성	손으로 그린 BPMN 5개(승인/구매/여행예약/논문심사/공급망 — 中山大 사례 세트 재현)가 모두 배포·실행
2 모델러	6-8	팔레트 제한, bc 속성 패널, 조건 빌더, 토큰 시뮬레이션, 한국어 오류 메시지	비개발자 3명이 설명 없이 "경비 승인" 을 그려서 컴파일 통과
3 런타임	9-12	배포 어댑터(Base Sepolia 또는 Kairos), 인덱서, 4개 화면, 임베디드 지갑 + 스폰서	3인 3역할로 인스턴스 10건 완주, 가스 0원 체감
4 확장	13+	L1 요소(타이머·결제·오라클), 버전 교체 투표, 신용서비스 프리셋, 논문화	—

각 Phase 끝에 **논문화 가능한 측정** 을 남긴다: 가스 표(6.8 형식), 검증 시간, 사용자 테스트 성공률. 中山大·杭电·哈工大 사례 세트와 같은 시나리오로 재면 비교표가 바로 나온다.

12. 리스크·오픈 이슈

리스크	영향	대응
1-safe 제약을 사용자가 이해 못 함 (루프·재진입 그리기)	검증 실패 반복	R3/R4 로 태스크 분기 금지, 루프는 L1 에서 "다시 하기" 패턴 프리셋으로만
조건식이 온체인 변수에만 의존 → 외부 데이터(환율 등) 불가	표현력	L1 오라클 서비스 태스크 (콜백 이벤트 + 서명된 응답)
프라이버시: 금액·결정이 퍼블릭 체인에 노출	실사용 제약	값 대신 커밋 해시 저장 옵션; CONFETTY(2026, MA-ABE) [읽음] 방향은 후속

리스크	영향	대응
bpmn-js 워터마크	상용화 시	그대로 유지(라이선스 준수). 자체 렌더러 교체는 비용 대비 낮은 우선 순위
스폰서 비용 폭주	운영비	9.4 정책 + 일일 한도
앵커 논문 본문 미확인 상태로 인용	논문화 시	CNKI/万方 또는 XJTU IR(item 484939) 에서 원문 확보 후 4.3 프리셋-2.1 절 보강
EIP-7702 위임 계약의 보안	자금	불변(immutable) CREATE2 위임 대상만, 초기화 서명 검증 [읽음]
인스턴스 256+ 플로우 프로세스	스케일	uint256[] marking 확장 (L2), 또는 서브프로세스 분할

부록 A. moddle 확장 JSON (bc.json)

```
{
  "name": "BlockFlow", "uri": "http://blockflow.dev/schema/bpmn/bc", "prefix": "bc",
  "xml": { "tagAlias": "lowerCase" },
  "types": [
    { "name": "Variable", "properties": [
      { "name": "name", "isAttr": true, "type": "String" },
      { "name": "type", "isAttr": true, "type": "String" },
      { "name": "initial", "isAttr": true, "type": "String" } ] },
    { "name": "Variables", "superClass": ["Element"], "properties": [
      { "name": "values", "isMany": true, "type": "Variable" } ] },
    { "name": "Input", "properties": [
      { "name": "variable", "isAttr": true, "type": "String" },
      { "name": "label", "isAttr": true, "type": "String" },
      { "name": "required", "isAttr": true, "type": "Boolean", "default": true } ] },
    { "name": "Inputs", "superClass": ["Element"], "properties": [
      { "name": "values", "isMany": true, "type": "Input" } ] },
    { "name": "LaneExt", "extends": ["bpmn:Lane"], "properties": [
      { "name": "roleKey", "isAttr": true, "type": "String" },
      { "name": "bindingMode", "isAttr": true, "type": "String", "default": "static" } ] },
    { "name": "TaskExt", "extends": ["bpmn:UserTask"], "properties": [
      { "name": "taskId", "isAttr": true, "type": "Integer" } ] }
  ]
}
```

bc:Variables 와 bc:Inputs 는 각각 bpmn:Process / bpmn:UserTask 의 extensionElements 자식으로 저장한다 (bpmn-js properties-panel-extension 예제의 패턴 [읽음]).

부록 B. IR 예시 (경비 승인, 전문)

```
{
  "version": "bf-ir/0.1",
  "process": { "id": "ExpenseApproval", "name": "경비 승인" },
  "roles": [
    { "key": "Requester", "label": "신청자", "binding": "static" },
    { "key": "Manager", "label": "팀장", "binding": "static" },
    { "key": "Finance", "label": "재무", "binding": "static" } ],
  "variables": [
    { "name": "amount", "type": "uint256" },
    { "name": "approved", "type": "bool" },
    { "name": "receiptHash", "type": "bytes32" } ],
  "flows": [
    { "id": "F1", "bit": 0, "from": "start", "to": "T1" },
    { "id": "F2", "bit": 1, "from": "T1", "to": "X1" },
    { "id": "F3", "bit": 2, "from": "X1", "to": "T2", "cond": "amount > 1000" },
    { "id": "F4", "bit": 3, "from": "X1", "to": "A1", "default": true },
    { "id": "F5", "bit": 4, "from": "T2", "to": "X2" },
    { "id": "F6", "bit": 5, "from": "A1", "to": "T3" },
    { "id": "F7", "bit": 6, "from": "A1", "to": "T4" },
    { "id": "F8", "bit": 7, "from": "T3", "to": "A2" },
  ]
}
```

```

    { "id": "F9", "bit": 8, "from": "T4", "to": "A2" },
    { "id": "F10", "bit": 9, "from": "A2", "to": "end_ok" },
    { "id": "F11", "bit": 10, "from": "X2", "to": "A1", "cond": "approved" },
    { "id": "F12", "bit": 11, "from": "X2", "to": "end_rej", "default": true } ],
  "nodes": [
    { "id": "start", "kind": "startEvent", "out": ["F1"] },
    { "id": "T1", "kind": "userTask", "taskId": 1, "name": "submit", "label": "경비 신청", "
role": "Requester", "inputs": [ { "variable": "amount", "label": "금액" } ], "in": ["F1"], "out": ["
F2"] },
    { "id": "X1", "kind": "xorSplit", "in": ["F2"], "branches": [ { "flow": "F3", "cond": "amount >
1000" } ], "default": "F4" },
    { "id": "T2", "kind": "userTask", "taskId": 2, "name": "approve", "label": "승인", "
role": "Manager", "inputs": [ { "variable": "approved", "label": "승인 여부" } ], "in": ["F3"], "out
": ["F5"] },
    { "id": "X2", "kind": "xorSplit", "in": ["F5"], "branches": [ { "flow": "F11", "cond": "approved
" } ], "default": "F12" },
    { "id": "A1", "kind": "andSplit", "in": ["F4", "F11"], "out": ["F6", "F7"] },
    { "id": "T3", "kind": "userTask", "taskId": 3, "name": "pay", "label": "지급", "
role": "Finance", "inputs": [], "in": ["F6"], "out": ["F8"] },
    { "id": "T4", "kind": "userTask", "taskId": 4, "name": "uploadReceipt", "label": "영수증 제출", "
role": "Requester", "inputs": [ { "variable": "receiptHash", "label": "영수증 파일" } ], "in":
["F7"], "out": ["F9"] },
    { "id": "A2", "kind": "andJoin", "in": ["F8", "F9"], "out": ["F10"] },
    { "id": "end_ok", "kind": "endEvent", "in": ["F10"], "outcome": "completed" },
    { "id": "end_rej", "kind": "endEvent", "in": ["F12"], "outcome": "rejected" } ],
  "silent": ["X1", "X2", "A1", "A2", "end_ok", "end_rej"]
}

```

부록 C. 생성 Solidity 전문 (경비 승인, solc 0.8.37 검증)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title ExpenseApproval - BPMN "경비 승인" 프로세스에서 자동 생성된 컨트랙트 (예시)
/// @notice 생성기 출력 형태를 보여주기 위한 참조 구현. 한 컨트랙트가 여러 인스턴스를 담는다.
/// @dev 상태 인코딩: 시퀀스 플로우 1개 = uint256 marking 의 비트 1개 (1-safe 가정)
contract ExpenseApproval {
    // ----- 역할 (BPMN 레인) -----
    bytes32 public constant ROLE_REQUESTER = keccak256("Requester");
    bytes32 public constant ROLE_MANAGER = keccak256("Manager");
    bytes32 public constant ROLE_FINANCE = keccak256("Finance");

    // ----- 시퀀스 플로우 비트 (생성기가 부여) -----
    uint256 private constant F1 = 1 << 0; // Start -> T1 submit
    uint256 private constant F2 = 1 << 1; // T1 -> X1 (XOR: amount>1000?)
    uint256 private constant F3 = 1 << 2; // X1 [yes] -> T2 approve
    uint256 private constant F4 = 1 << 3; // X1 [default] -> A1 (AND split)
    uint256 private constant F5 = 1 << 4; // T2 -> X2 (XOR: approved?)
    uint256 private constant F6 = 1 << 5; // A1 -> T3 pay
    uint256 private constant F7 = 1 << 6; // A1 -> T4 uploadReceipt
    uint256 private constant F8 = 1 << 7; // T3 -> A2 (AND join)
    uint256 private constant F9 = 1 << 8; // T4 -> A2
    uint256 private constant F10 = 1 << 9; // A2 -> End
    uint256 private constant F11 = 1 << 10; // X2 [yes] -> A1
    uint256 private constant F12 = 1 << 11; // X2 [default] -> End (반려)

    // ----- 태스크 식별자 (이벤트/UI 용) -----
    uint8 public constant TASK_SUBMIT = 1;
    uint8 public constant TASK_APPROVE = 2;
    uint8 public constant TASK_PAY = 3;
    uint8 public constant TASK_RECEIPT = 4;

    // ----- 프로세스 변수 (BPMN dataFields 확장에서 생성) -----
    struct Vars {
        uint256 amount;
        bool approved;
        bytes32 receiptHash; // 오프체인 문서는 해시만 저장
    }

    struct Instance {

```

```

    uint256 marking;
    bool ended;
    address creator;
}

// ===== 저장소 =====
address public owner;           // 프로세스 소유자(비전문가 계정 또는 그 스마트 계정)
bool public paused;
uint256 public instanceCount;
mapping(uint256 => Instance) public instances;
mapping(uint256 => Vars) public vars;
mapping(uint256 => mapping(bytes32 => address)) public roleOf; // instance -> role -> account

// ===== 이벤트 (오프체인 인덱서/UI 가 구독) =====
event InstanceCreated(uint256 indexed id, address indexed creator);
event RoleBound(uint256 indexed id, bytes32 indexed role, address account);
event TaskCompleted(uint256 indexed id, uint8 indexed taskId, address indexed actor);
event MarkingChanged(uint256 indexed id, uint256 marking);
event InstanceEnded(uint256 indexed id, bool completed); // completed=false 이면 반려 종료
event Paused(bool paused);

// ===== 커스텀 에러 (가스 절약 + UI 가 해석) =====
error NotOwner();
error IsPaused();
error NotAuthorized(uint256 id, bytes32 role);
error TaskNotEnabled(uint256 id, uint8 taskId);
error AlreadyEnded(uint256 id);
error RoleCount();

constructor(address _owner) {
    owner = _owner;
}

modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

modifier whenNotPaused() {
    if (paused) revert IsPaused();
    _;
}

modifier onlyRole(uint256 id, bytes32 role) {
    if (roleOf[id][role] != msg.sender) revert NotAuthorized(id, role);
    _;
}

/// @dev 태스크 진입 가드: 인스턴스 미종료 + 입력 플로우 토큰 보유
function _require(uint256 id, uint256 inFlow, uint8 taskId) internal view {
    Instance storage inst = instances[id];
    if (inst.ended) revert AlreadyEnded(id);
    if (inst.marking & inFlow == 0) revert TaskNotEnabled(id, taskId);
}

// ===== 소유자 제어 =====
function setPaused(bool p) external onlyOwner {
    paused = p;
    emit Paused(p);
}

/// @notice 인스턴스 생성 + 역할 바인딩 (역할 순서: Requester, Manager, Finance)
function createInstance(address[3] calldata roleAccounts)
    external
    whenNotPaused
    returns (uint256 id)
{
    id = ++instanceCount;
    Instance storage inst = instances[id];
    inst.creator = msg.sender;
    inst.marking = F1; // 시작 이벤트가 첫 플로우에 토큰을 놓는다

    bytes32[3] memory roles = [ROLE_REQUESTER, ROLE_MANAGER, ROLE_FINANCE];

```

```

    for (uint256 i = 0; i < 3; i++) {
        if (roleAccounts[i] == address(0)) revert RoleCount();
        roleOf[id][roles[i]] = roleAccounts[i];
        emit RoleBound(id, roles[i], roleAccounts[i]);
    }
    emit InstanceCreated(id, msg.sender);
    emit MarkingChanged(id, F1);
}

/// @notice 소유자가 역할 담당자를 교체 (담당자 이탈 대비)
function rebindRole(uint256 id, bytes32 role, address account) external onlyOwner {
    roleOf[id][role] = account;
    emit RoleBound(id, role, account);
}

// ————— 사용자 태스크 (BPMN userTask 1개 = 함수 1개) —————

/// T1: 경비 신청 [Requester] in: F1 out: F2 sets: amount
function submit(uint256 id, uint256 amount)
    external
    whenNotPaused
    onlyRole(id, ROLE_REQUESTER)
{
    _require(id, F1, TASK_SUBMIT);
    vars[id].amount = amount;
    _fire(id, F1, F2, TASK_SUBMIT);
}

/// T2: 승인 [Manager] in: F3 out: F5 sets: approved
function approve(uint256 id, bool approved)
    external
    whenNotPaused
    onlyRole(id, ROLE_MANAGER)
{
    _require(id, F3, TASK_APPROVE);
    vars[id].approved = approved;
    _fire(id, F3, F5, TASK_APPROVE);
}

/// T3: 지급 [Finance] in: F6 out: F8
function pay(uint256 id)
    external
    whenNotPaused
    onlyRole(id, ROLE_FINANCE)
{
    _require(id, F6, TASK_PAY);
    _fire(id, F6, F8, TASK_PAY);
}

/// T4: 영수증 제출 [Requester] in: F7 out: F9 sets: receiptHash
function uploadReceipt(uint256 id, bytes32 receiptHash)
    external
    whenNotPaused
    onlyRole(id, ROLE_REQUESTER)
{
    _require(id, F7, TASK_RECEIPT);
    vars[id].receiptHash = receiptHash;
    _fire(id, F7, F9, TASK_RECEIPT);
}

// ————— 엔진 —————

/// @dev 토큰 소비/생산 후 자동 전이(게이트웨이)를 고정점까지 실행
function _fire(uint256 id, uint256 consume, uint256 produce, uint8 taskId) internal {
    Instance storage inst = instances[id];
    uint256 m = (inst.marking & ~consume) | produce;
    emit TaskCompleted(id, taskId, msg.sender);
    m = _step(id, m);
    inst.marking = m;
    emit MarkingChanged(id, m);
}

/// @dev 게이트웨이/종료 이벤트 = 외부 입력 없이 진행 가능한 "침묵 전이"

```

```

function _step(uint256 id, uint256 m) internal returns (uint256) {
    Vars storage v = vars[id];
    bool progressed = true;
    while (progressed) {
        progressed = false;

        // X1: XOR split in: F2 out: F3 [amount > 1000] | F4 [default]
        if (m & F2 != 0) {
            m &= ~F2;
            m |= (v.amount > 1000) ? F3 : F4;
            progressed = true;
        }

        // X2: XOR split in: F5 out: F11 [approved] | F12 [default]
        if (m & F5 != 0) {
            m &= ~F5;
            m |= v.approved ? F11 : F12;
            progressed = true;
        }

        // A1: AND split in: F4 | F11 out: F6 & F7
        if (m & (F4 | F11) != 0) {
            m &= ~(F4 | F11);
            m |= (F6 | F7);
            progressed = true;
        }

        // A2: AND join in: F8 & F9 out: F10
        if (m & (F8 | F9) == (F8 | F9)) {
            m &= ~(F8 | F9);
            m |= F10;
            progressed = true;
        }

        // End (정상 종료) in: F10
        if (m & F10 != 0) {
            m &= ~F10;
            instances[id].ended = true;
            emit InstanceEnded(id, true);
            progressed = true;
        }

        // End (반려 종료) in: F12
        if (m & F12 != 0) {
            m &= ~F12;
            instances[id].ended = true;
            emit InstanceEnded(id, false);
            progressed = true;
        }
    }
    return m;
}

// ————— 조회 (UI 용) —————

/// @notice 현재 실행 가능한 태스크 비트맵 (bit = taskId)
function enabledTasks(uint256 id) external view returns (uint256 bits) {
    uint256 m = instances[id].marking;
    if (instances[id].ended) return 0;
    if (m & F1 != 0) bits |= 1 << TASK_SUBMIT;
    if (m & F3 != 0) bits |= 1 << TASK_APPROVE;
    if (m & F6 != 0) bits |= 1 << TASK_PAY;
    if (m & F7 != 0) bits |= 1 << TASK_RECEIPT;
}
}

```

부록 D. 자동 생성 Foundry 테스트 예시

생성기는 IR에서 아래 형태의 테스트를 만든다. (forge test --match-contract ExpenseApproval)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
import "forge-std/Test.sol";
import "../src/ExpenseApproval.sol";

```

```

contract Handler is Test {
    ExpenseApproval p; address[3] acc; uint256 constant N = 3;
    constructor(ExpenseApproval _p, address[3] memory _acc) { p = _p; acc = _acc; }
    function create() external { p.createInstance(acc); }
    function submit(uint256 id, uint256 amt, uint8 who) external {
        id = bound(id, 1, p.instanceCount() + 1); vm.prank(acc[who % 3]);
        try p.submit(id, amt) {} catch {}
    }
    function approve(uint256 id, bool ok, uint8 who) external {
        id = bound(id, 1, p.instanceCount() + 1); vm.prank(acc[who % 3]);
        try p.approve(id, ok) {} catch {}
    }
    function pay(uint256 id, uint8 who) external {
        id = bound(id, 1, p.instanceCount() + 1); vm.prank(acc[who % 3]);
        try p.pay(id) {} catch {}
    }
    function uploadReceipt(uint256 id, bytes32 h, uint8 who) external {
        id = bound(id, 1, p.instanceCount() + 1); vm.prank(acc[who % 3]);
        try p.uploadReceipt(id, h) {} catch {}
    }
}

contract ExpenseApprovalInvariants is Test {
    ExpenseApproval p; Handler h;
    uint256 constant ALL_FLOWS = (1 << 12) - 1; // IR flows 집합 (생성기가 채움)

    function setUp() public {
        address[3] memory acc = [address(0xA1), address(0xB2), address(0xC3)];
        p = new ExpenseApproval(address(this));
        h = new Handler(p, acc);
        targetContract(address(h));
    }

    /// 모든 인스턴스의 marking 은 알려진 플로우 비트만 갖는다
    function invariant_noGhostTokens() public view {
        for (uint256 id = 1; id <= p.instanceCount(); id++) {
            (uint256 m,,) = p.instances(id);
            assertEq(m & ~ALL_FLOWS, 0);
        }
    }

    /// 종료된 인스턴스는 토큰이 없다 (proper completion)
    function invariant_endedMeansEmpty() public view {
        for (uint256 id = 1; id <= p.instanceCount(); id++) {
            (uint256 m, bool ended,) = p.instances(id);
            if (ended) assertEq(m, 0);
        }
    }

    /// 1-safe: AND 분기 뒤 두 가지(F6,F7) 는 동시에 있을 수 있지만, XOR 양 가지(F3,F4) 는 동시에 없다
    function invariant_xorExclusive() public view {
        for (uint256 id = 1; id <= p.instanceCount(); id++) {
            (uint256 m,,) = p.instances(id);
            assertTrue((m & (1 << 2)) == 0 || (m & (1 << 3)) == 0);
        }
    }
}

```

foundry.toml :

```

[invariant]
runs = 256
depth = 64
fail_on_revert = false

```

V2 의 BFS 가 만든 도달 경로마다 시나리오 테스트(test_path_<n>)를 추가로 생성해 경로 커버리지를 100% 로 만든다.

부록 E. 용어 대역표 (中 / 英 / 韓)

中文	English	한국어	비고
业务流程模型与标注 (BPMN)	Business Process Model and Notation	업무 프로세스 모델·표기	
编排图	choreography diagram	코레오그래피 다이어그램	杭电·哈工大 논문의 핵심 용어
协作图	collaboration diagram	협업 다이어그램	中山大
泳道	lane / swimlane	레인	본 도구의 역할 단위
排他网关 / 并行网关 / 包容网关	exclusive / parallel / inclusive gateway	배타 / 병렬 / 포괄 게이트웨이	
事件网关	event-based gateway	이벤트 기반 게이트웨이	L1
顺序流 / 消息流	sequence flow / message flow	시퀀스 플로우 / 메시지 플로우	
用户任务 / 服务任务 / 脚本任务	user / service / script task	사용자 / 서비스 / 스크립트 태스크	
业务规则任务	business rule task	비즈니스 규칙 태스크	BlockCollab 의 DMN 호출
子流程 / 调用活动	subprocess / call activity	서브프로세스 / 콜 액티비티	L2
多实例	multi-instance	다중 인스턴스	哈工大 2025
智能合约	smart contract	스마트 컨트랙트	
链码	chaincode	체인코드	Fabric 용어
状态机 / 有限状态机	(finite) state machine	(유한) 상태기계	FSM-SCG
状态图	statecharts	스테이트차트	IBM 2018, 哈工大 2025
Petri网 / 标识 / 变迁 / 库所	Petri net / marking / transition / place	페트리넷 / 마킹 / 전이 / 플레이스	본 문서 5.3, 7.2
位图 / 位掩码	bitmap / bitmask	비트맵 / 비트마스크	D1
模型检测	model checking	모델 체크	中山大 CSP#/PAT
死锁 / 不可达	deadlock / unreachable	데드락 / 도달 불가	
形式化验证	formal verification	형식 검증	
中间表示	intermediate representation (IR)	중간표현	5장
模板 / 代码生成	template / code generation	템플릿 / 코드 생성	6.7
低代码 / 无代码 / 可视化	low-code / no-code / visual	로우코드 / 노코드 / 시각적	특허 T2
非专业人员	non-expert	비전문가	
角色绑定 / 动态绑定	role binding / dynamic binding	역할 바인딩 / 동적 바인딩	D5
基于属性的访问控制 (ABAC)	attribute-based access control	속성 기반 접근제어	BlockCollab
预言机	oracle	오라클	L1
链上 / 链下	on-chain / off-chain	온체인 / 오프체인	
燃料费 / Gas 费	gas fee	가스비	
手续费代付	fee delegation	수수료 대납	Kaia
账户抽象	account abstraction	계정 추상화	ERC-4337 / EIP-7702
大模型	large (language) model	대규모 언어모델	D8

中文	English	한국어	비고
范式	paradigm	패러다임 / 범식	앵커 논문 제목
信用服务	credit service	신용 서비스	앵커 논문 도메인

부록 F. 참고문헌 (검증 상태 포함)

F.1 중국어권 논문·특허

#	서지	검증	이 문서에서의 쓰임
C1	薄钧戈. 面向信用服务的BPMN智能合约范式研究. 信息安全研究, 2022, 8(5): 475–483. 西安交通大学. http://www.sicris.cn/CN/abstract/abstract1167.shtml · XJTU IR item 484939	서지만 [미검증]	앵커. 4.3 도메인 프리셋
C2	SmartGen-AADL. 软件学报, 2026 第8期 "大模型与软件工程" 特辑 우선출판. https://www.jos.org.cn/	특집 게재만 [미검증]	D8
C3	Jin J., Yan L., Zou Y., Li J., Yu Z. (北京物资学院·北航). Research on Smart Contract Verification and Generation Method Based on BPMN. Mathematics 12(14):2158, 2024. https://www.mdpi.com/2227-7390/12/14/2158	[읽음]	R1~R12, D9
C4	Xiong T., Feng S., Pan M., Yu Y. (中山大学). Smart Contract Generation for Inter-Organizational Process Collaboration. arXiv:2303.09257; CCPE 2024. https://arxiv.org/abs/2303.09257 · https://github.com/xthHub/SCG4IOPC	[읽음]	D4, D9, 11장 사례 세트
C5	Feng S., Jia C., Pan M., Yu Y. (中山大学). Smart Contract Generation Supporting Multi-instance for IOPC. ChineseCSCW 2023, CCIS 2013. DOI 10.1007/978-981-99-9640-7_29	[초록]	L2
C6	俞东进, 魏一杰, 孙笑笑, 倪可, 沈虎军 (杭州电子科技大学). 编排图驱动的区块链业务过程管理框架. 通信学报 42(9):121–132, 2021. DOI 10.11959/j.issn.1000-436x.2021171	[읽음]	D2, D3, C6 버전 투표
C7	Shen X., Luo J., Wang H., Liu M., Dustdar S., Wang Z. (哈尔滨工业大学). A Hybrid BPMN-DMN Framework ... (BlockCollab). arXiv:2412.01196, 2024. https://github.com/XinzheShen182/ChainCollab	[읽음]	D5, D7, L2
C8	Shen X. et al. (哈工大). A Statecharts-Based Model-Driven Approach ... Multiple Instances. Blockchain: Research and Applications, 2025. DOI 10.1016/j.bcr.2025.100421 · https://github.com/HIT-ICES/ChainCollab	[초록]	L2 다중 인스턴스
C9	Luo H. et al. (武汉大学). Guiding LLM-based Smart Contract Generation with Finite State Machine (FSM-SCG). arXiv:2505.08542, 2025	[읽음]	D8
C10	高一琛, 赵斌, 张召 (华东师大). 面向以太坊的智能合约自动生成方法研究与实现. 华东师范大学学报(自然科学版) 2020(5):21–32	[초록]	비전문가 지향 초기 연구
C11	王迪, 朱岩 et al. (北京科技大学). 智能法律合约及其研究进展. 工程科学学报 44(1):68–81, 2022	[초록]	고급 계약 언어 계보
C12	中央财经大学 特许 CN117234488B. 基于EPC模型的智能法律合约生成方法及装置 (高胜, 刘文婷, 朱建明), 2024	[읽음]	D12
C13	墨石科技 特许 CN112148278A. 可视化的区块链智能合约框架及智能合约开发部署方法, 2020	[읽음]	D12, 4.3
C14	平安国际智慧城市 特许 CN112015396A. 基于DSL的智能合约代码生成方法, 2020	[읽음]	D12, 6.7
C15	计算机集成制造系统 관련: 基于区块链的业务流程互操作服务框架 (abstract7154); 基于赋时编排图的区块链业务流程管理引擎 (abstract7787); 基于区块链的跨组织业务流程模型演化和实例迁移 (abstract8882)	제목만 [미검증]	C6 버전 교체 참고
C16	FISCO BCOS WeBASE 可视化合约IDE 문서	[읽음]	저코드 참고 (BPMN 없음)

F.2 영어권 논문

#	서지	검증	쓰임
E1	Weber I. et al. Untrusted Business Process Monitoring and Execution Using Blockchain. BPM 2016	[읽음]	D5, D7, L1 결제
E2	García-Bañuelos L., Ponomarev A., Dumas M., Weber I. Optimized Execution of Business Processes on Blockchain. BPM 2017. arXiv:1612.03152	[읽음]	D1, 5.2 축약
E3	López-Pintado O. et al. Caterpillar: A Business Process Execution Engine on the Ethereum Blockchain. SPE 2019. arXiv:1808.03517; Interpreted Execution ... EDOC 2019. arXiv:1906.01420; https://github.com/orlenyslp/Caterpillar	[읽음]	D2, 6.3
E4	López-Pintado O. et al. Dynamic Role Binding in Blockchain-Based Collaborative Business Processes. CAiSE 2019	[초록 +세부]	D5
E5	Tran A.B., Lu Q., Weber I. Lorikeet. BPM 2018 Demo. https://ceur-ws.org/Vol-2196/BPM_2018_paper_12.pdf	[읽음]	자산 레지스트리(L2)
E6	Corradini F. et al. ChorChain. SAC 2020 / BPM 2022 Demo; FlexChain. FGCS 147, 2023	[읽음]	D2, D3, D4
E7	Sturm C. et al. A Lean Architecture for Blockchain Based Decentralized Process Execution. BPM 2018 WS	[읽음]	해석형 참고
E8	Stiehle F., Weber I. Blockchain for Business Process Enactment: A Taxonomy and SLR. BPM 2022; Chorpiller https://github.com/fstiehle/chorpiller	[읽음]	D4, 6.7
E9	Stiehle F., Weytjens H., Weber I. On LLM-Assisted Generation of Smart Contracts from Business Processes. 2025. arXiv:2507.23087	[읽음]	D8
E10	Tonga Naha R., Zhang K. Pupa. BPM 2022	[초록]	L1 타이머 · OR
E11	Atwi, Pautasso. BPMN4BC. ICSOC 2025	[초록]	on/off-chain 표기
E12	Liu, Bodorik, Jutla. BPMN Model to Smart Contract by Business Analyst. arXiv:2505.22612	[읽음]	DE-HSM, DMN
E13	Nakamura, Miyamoto, Kudo. Inter-organizational Business Processes Managed by Blockchain. WISE 2018	[초록]	스테이트차트
E14	CONFETTY. BPM Forum 2026. arXiv:2603.13900	[읽음]	12장 프라이버시
E15	Evaluating the Vulnerability Landscape of LLM-Generated Smart Contracts. arXiv:2602.04039	[읽음]	D8

F.3 도구·인프라 (2026-09 검증)

bpmn-js 18.28 / bpmn-moddle 10.2 / properties-panel / token-simulation 0.40 / form-js 1.26 (npm) · Solidity 0.8.37 릴리스 노트 · OpenZeppelin 5.6.1 · Foundry 1.8.3 · Hardhat 3.16 · viem 2.56 / wagmi 3.7 · ethereum.org Pectra/EIP-7702 · eth-infinitism EntryPoint v0.7/0.8/0.9 · permissionless.js 0.4 · Privy pricing · Base Account docs · Kaia fee delegation / Fee Delegation Service / Gas Abstraction docs · Slither 0.11.6 · solc SMTChecker docs · Foundry invariant testing docs · arXiv 2606.22206 (L2 수수료 측정).